

Fortgeschrittene Informatik

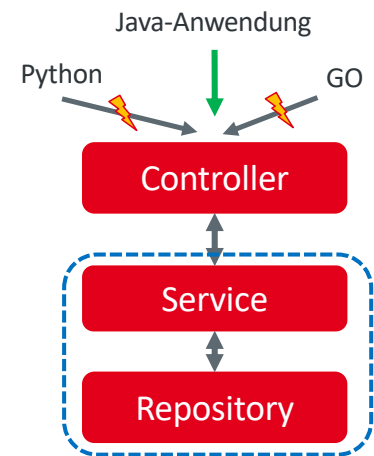
Prof. Dr. Andreas Schilling

www.ravensburg.dhbw.de

3. REST

Motivation

- **Status-Quo:** Unsere Webanwendung einen **Zustand in der Datenbank vorhalten**, der über die Servicemethoden verändert werden kann.
- Die **Controllermethoden** können nur von **anderen Java-Anwendungen** aufgerufen werden, was deren Einsatzmöglichkeit reduziert.
- Wie können die **Controllermethoden unabhängig von der Programmiersprache** der Aufrufer verwendet werden?

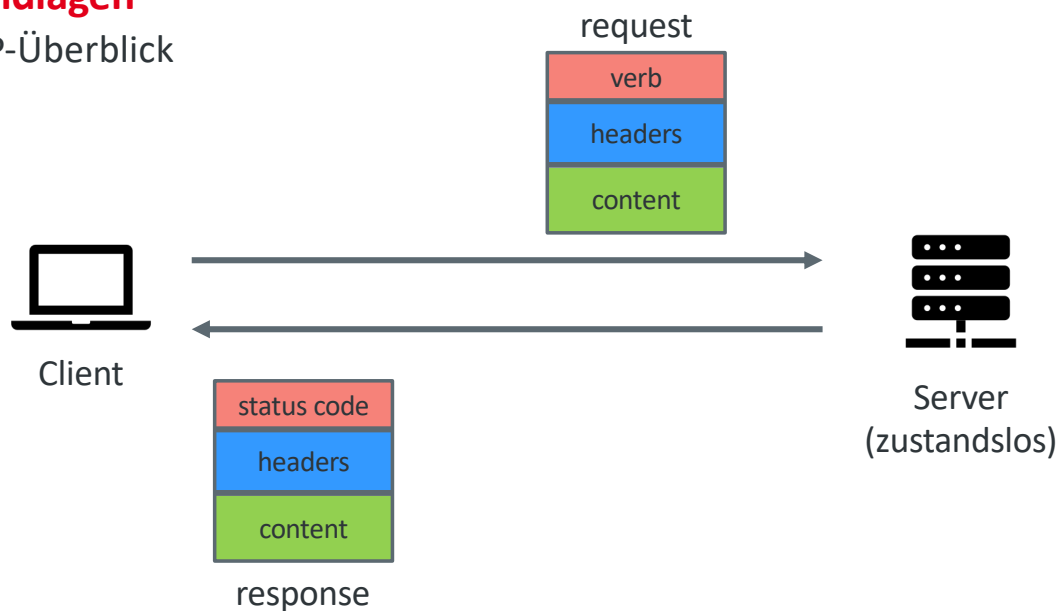


Kap. 2



Grundlagen

HTTP-Überblick

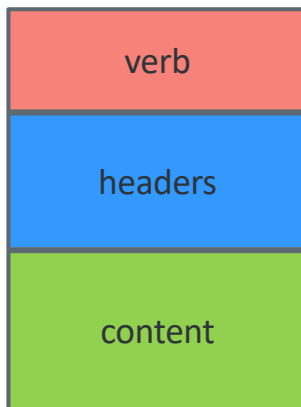


Das HTTP-Protokoll beschreibt den Informationsaustausch zwischen einem Client und einem zustandslosen Server. Im Rahmen des HTTP-Request fragt der Client bestimmte Ressourcen vom Server ab. Dieser HTTP-Request besteht aus drei Teilen: Das verwendete HTTP-Verb beschreibt den Anfragetyp, also bspw. ob es sich bei der Anfrage um ein Informationsabfrage (bspw. GET) oder eine Informationsübergabe (bspw. POST) handelt. Der HTTP-Header beinhaltet Steuerinformationen für die Übermittlung und Verarbeitung der Anfrage (bspw. Größe der Anfrage). Der letzte, optionale, Teil der Anfrage ist der Inhaltsteil über den weitere Informationen übergeben werden.

Die HTTP-Antwort besteht ebenfalls aus drei Teilen. Der erste Teil ist der Status Code, der über den Verarbeitungszustand der Anfrage auf dem Server informiert. Der HTTP-Header beinhaltet wieder Steuerinformationen für die Übermittlung und Interpretation der Antwort. Zuletzt besteht auch die Antwort aus einem Inhaltsteil, der die Beantwortung der Anfrage umfasst.

Grundlagen

HTTP Anfrage



HTTP Verb (auszuführende Aktion):

- **GET:** Ruft eine Kopie der angegebenen Ressource auf
- **POST:** Erstellt eine neue Ressource
- **PUT:** Aktualisiert die adressierte Ressource.
- **PATCH:** Partielle Änderung einer Ressource.
- **DELETE:** Entfernt die adressierte Ressource.

HTTP Header:

- **Content-Type:** Das Format der Anfrage
- **Content-Length:** Die Größe der Anfrage
- **Accept:** Welche Antwortform wird akzeptiert

HTTP Content/Payload (optional):

- Strukturierte Anfrage (JSON, XML, Text)
- Binärdatei

Schauen wir uns im Folgenden genauer den Aufbau der HTTP-Anfrage an. Der erste Teil der Anfrage besteht aus dem HTTP-Verb, das den Anfragetyp charakterisiert. Hierbei werden folgende fünf Anfragen unterschieden:

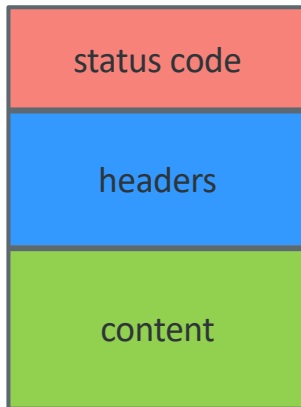
- GET: Fragt eine Kopie der angegebenen Ressource an.
- POST: Erstellt eine neue Ressource basierend auf dem übergebenen Inhalt.
- PUT: Aktualisiert eine bestehende Ressource mit dem übergebenen Inhalt.
- PATCH: Aktualisiert eine Ressource. Im Vergleich zu PUT werden hier jedoch nur die Änderungen mitgegeben.
- DELETE: Anfrage um eine Ressource auf dem Server zu löschen.

Der HTTP-Header beinhaltet Steuerinformationen zur Weiterleitung und Verarbeitung der Anfrage wie beispielsweise das Format der Anfrage, die Größe der Anfrage und welche Formate vom Client für eine Antwort unterstützt werden.

Im Falle von POST, PUT und PATCH Anfragen besteht der optionale Inhaltsteil der Anfrage aus strukturierten Informationen. Neben strukturierten Informationen können aber auch Binärdateien im Inhaltsteil übertragen werden.

Grundlagen

HTTP Antwort



Status Code (Mitteilung über die Ausführung):

- **100-199:** Informationsmitteilung
- **200-299:** Erfolgsmitteilungen
- **300-399:** Weiterleitung
- **400-499:** Client-Fehler
- **500-599:** Server-Fehler

HTTP Header:

- **Content-Type:** Das Format der Antwort
- **Content-Length:** Die Größe der Antwort
- **Expires:** Gültigkeit der Antwort.

HTTP Content:

- Strukturierte Antwort (HTML, CSS, JS, etc.)
- Binärdatei

Die Antwort auf eine HTTP-Anfrage besitzt ebenfalls einen dreigeteilten Aufbau. Der erste Teil der Antwort, der Status Code, informiert über den Verarbeitungszustand der Anfrage auf dem Server. Die einzelnen Status Codes sind in die folgenden Klassen eingeteilt um ihre Interpretation zu erleichtern.

- 100-199: Die Bearbeitung der Anfrage dauert noch an und es wird ggf. ein anderes Protokoll verwendet (vgl. 101).
- 200-299: Die Bearbeitung der Anfrage war erfolgreich und hat ggf. eine neue Ressource angelegt (vgl. 201).
- 300-399: Für die Bearbeitung der Anfrage wird der Client an einen anderen Server verwiesen (vgl. 301).
- 400-499: Die HTTP-Anfrage war fehlerhaft. Bspw. wurden Authentifizierungsinformationen nicht übertragen (vgl. 401).
- 500-599: Auf dem Server ist bei der Bearbeitung der Anfrage ein Fehler aufgetreten. Bspw. Server nicht verfügbar (vgl. 503).

Der zweite Teil der Anfrage besteht aus dem HTTP-Header, der Informationen zur Weiterleitung und Verarbeitung beinhaltet. Typische Header-Informationen einer HTTP-Antwort sind die Beschreibung des Typs der übertragenen Informationen im Inhaltsteil, der Größe der Antwort und eine optionale Angabe wie lange die Antwort gültig ist.

Der letzte Teil der HTTP-Antwort ist der Inhaltsteil der einen strukturierten Aufbau aufweisen kann, wenn beispielsweise HTML übertragen wird, aber auch Binärdateien umfassen kann (bspw. im Falle eines File-Downloads).

Grundlagen

Austauschformate - JSON

- **JSON** (JavaScript Object Notation) ist ein Datenformat, das zur **Übertragung von strukturierten Daten** bei HTTP-Anfragen eingesetzt wird
- JSON verwendet für die Beschreibung von Objekten eine **an Java angelehnte Syntax**.
- **Geschweifte Klammern** beschreiben **Objekte** samt ihrer Attribute und Objektbeziehungen.
- **Eckige Klammern** repräsentieren **Arrays**.
- Die Attribute werden in der „**Schlüssel: Wert**“ Notation spezifiziert.

```

1  {
2      "ParkingEntities": [
3          {
4              "LicensePlate": "RV-RT-324",
5              "Model": "VW-Golf",
6              "Owner": {
7                  "Forname": "Hans",
8                  "Surname": "Schmidt",
9                  "Phone": 12321
10             }
11         }
12     ]
13 }

```

Eine gängiges Austauschformat für HTTP-Anfragen bei Web-Anwendungen ist die JavaScript Object Notation (JSON). JSON ermöglicht eine vereinfachte Lesbarkeit für Menschen, da die verwendete Syntax stark an jene der JAVA-Programmiersprache angelehnt ist. So verwendet JSON geschweifte Klammern um Objekt-Hierarchien abzubilden und eckige Klammern um Arrays zu repräsentieren. Die Attribute werden dabei in der Schlüssel-Wert Notation beschrieben. Dies bedeutet, dass zunächst der Attributname in Anführungszeichen geschrieben wird gefolgt von einem Doppelpunkt und dem Attributwert. Ist der Attributwert eine Zeichenkette wird diese in Anführungszeichen geschrieben, bei Zahlen wird direkt die Zahl geschrieben und im Falle von Objekten oder Arrays werden die Zeichen {} bzw. [] zur Beschreibung des Attributwerts eingesetzt.

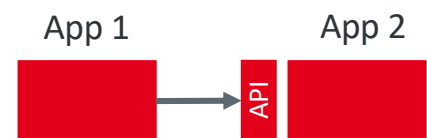
Der Vorteil von JSON gegenüber anderen Auszeichnungssprachen ist die Leichtigkeit, die dazu führt, dass Daten sowohl von Menschen als auch Maschinen sehr schnell interpretiert werden können. Ferner wird die benötigte Speichergröße der Objekte im Vergleich zu anderen Austauschformaten stark reduziert.



REST

Grundlagen

- Ein **Application Programming Interface (API)** ist eine Schnittstelle, die zur Kommunikation zwischen Anwendungen eingesetzt wird.
- Die API spezifiziert die unterstützten **Datentypen** und die angebotenen **Funktionen der Schnittstelle**.
- Um eine große **Wiederverwendung** einer Anwendung sicherzustellen sollte die API **sprachunabhängig** sein.
- APIs, die über das Internet angesprochen werden (**Web-APIs**), haben massiv zum Erfolg vieler Webdienste beigetragen.

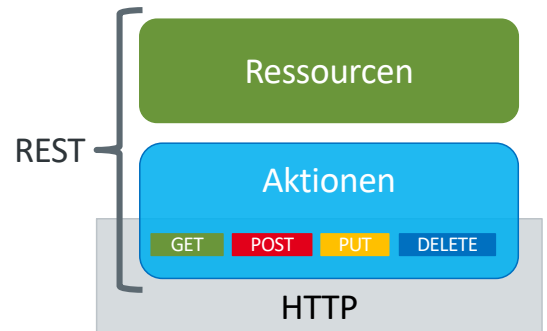


Ein Application Programming Interface (API) ist eine Softwareschnittstelle, die es ermöglicht, dass unterschiedliche Anwendungen miteinander kommunizieren können. Hierzu abstrahiert die Schnittstelle die internen Wirkzusammenhänge der Anwendung und bietet nach außen hin generische Funktionen und Datentypen an. Infolgedessen müssen Anwendungen, welche die Schnittstelle nutzen, minimales Vorwissen mitbringen. Um die Entkopplung der aufrufenden und gerufenen Anwendung noch weiter auszubauen ist es vorteilhaft die API sprachunabhängig umzusetzen, sodass die aufrufende und die aufgerufene Anwendung in unterschiedlichen Programmiersprachen verfasst werden können. Mit dem Paradigmenwechsel weg von allumfassenden Softwaresystemen hin zu spezialisierten Web-Anwendungen, die zur Erfüllung eines Geschäftsvorfalles miteinander interagieren, hat die Bedeutung von APIs in den letzten Jahren stark zugenommen.

REST

Grundlagen

- Representational State Transfer (**REST**) ist ein Paradigma zum Entwurf von Web-Schnittstellen, das von Roy Fielding entwickelt wurde.
- Das Ziel von REST ist es **langlebige** und **entkoppelte Interaktionen** umzusetzen.
- REST **baut auf HTTP** auf und unterscheidet zwischen **Ressourcen** und deren **Aktionen**.
- **Ressourcen** sind Objekte mit denen der Nutzer interagiert.
- **Aktionen** entsprechen fachlichen Anwendungsfällen, die auf den Ressourcen ausgeführt werden. Die Aktionen werden durch die unterschiedlichen HTTP-Anfragetypen aufgerufen.



11

Bei REST (Representational State Transfer) handelt es sich um ein Paradigma, das häufig zum Entwurf von Web-Schnittstellen eingesetzt wird. REST wurde von Roy Fielding, einem der Hauptautoren der HTTP-Spezifikation, im Rahmen seiner Dissertation entworfen. Das Ziel von REST besteht im Entwurf von Schnittstellen, die eine langlebige und entkoppelte Interaktion zwischen den Teilnehmern umsetzen. Ein Kernmerkmal von REST ist, dass es sich zur Umsetzung von Web-Schnittstellen sehr stark am HTTP-Protokoll orientiert. In Folge dessen ist REST ein sehr leichtgewichtiges Interaktionsprotokoll mit minimalem Overhead.

Der Grundgedanke von REST ist, dass es Ressourcen gibt auf denen Aktionen ausgeführt werden. Die Ressourcen werden durch Internetadressen (URLs) identifiziert und die Aktionen werden über die unterschiedlichen Anfragetypen des HTTP-Protokolls abgebildet.

REST

REST-Ressource

- Eine **REST-Ressource** wird via **URL identifiziert** (Protokoll & weltweit eindeutige Identifikation des Endpunkts).
- Der **Name** der REST-Ressource ist stets im **Plural**, selbstbeschreibend und beinhaltet **keine Verben**.
- **Geschachtelte Ressourcen** sind **zu vermeiden**. Stattdessen sollte auf andere Ressourcen verwiesen werden.
- **Links** sind Verweise auf Ressourcen.
- Es wird nicht auf die Ressource, sondern auf die **Repräsentationen** dieser zugegriffen.

http://web-corp.com/apis/v1/orders

Protokoll Domain Kontext-Pfad Ressource

- ✗ web-corp.com/apis/v1/cust/1/orders/2
- ✗ http://web-corp.com/
- ✗ http://web-corp.com/apis/v1/get-orders/1
- ✗ http://web-corp.com/apis/v1/cust/1/orders/2

Die URL ist wie ein technischer Schlüssel, der eine Web-Ressource weltweit eindeutig identifiziert. Eine URL besteht aus 4 Teilen: Als erstes wird das Protokoll spezifiziert, das zur Ansprache der Ressource eingesetzt wird. Nach dem Protokoll bis zum ersten Schrägstrich (eng. „Slash“) wird die Domain der Ressource spezifiziert. Im Anschluss daran wird der Kontext-Pfad auf dem Server zur Webressource definiert. Der letzte Teil der URL entspricht dem Namen der aufgerufenen Ressource.

Beim Verfassen von REST-APIs gibt es wichtige Richtlinien die beim Verfassen von URLs zu berücksichtigen sind. Die erste Regel ist, dass der Ressourcename stets im Plural verfasst wird und keine Funktionsbeschreibung beinhaltet (bspw. „orders“ und nicht „get-order“). Des Weiteren sollten geschachtelte Ressourcennamen stets vermieden werden, da sie die Identifikation welche Ressource aktuell aufgerufen wird behindern. Statt der Verschachtelung von Ressourcen sollten Links verwendet werden um auf andere Ressourcen zu verweisen. Abschließend gilt es zu beachten, dass bei Anfragen nicht die „echte“ Ressource vom Server zurückgegeben wird sondern stets eine Kopie dieser, sodass auf dem Server stets die „single point of truth“ der Ressource verbleibt.

REST

Aktionen

- Auszuführende Aktionen sind **self-contained**, das heißt der Server behält keinen Zustand über die Anfragen des Clients hinweg.
 - Durch Nutzung der HTTP-Anfragetypen, entsteht ein **generisches Interface** für die Veränderung der Ressourcen.
 - GET-Aufrufe können **zwischengespeichert** werden um die Anfragelast zu reduzieren.
 - GET, PUT und DELETE Aufrufe sind **idempotent**.
- | | |
|--------|------------------------------|
| GET | Informationsbezug |
| POST | Erstellung neuer Ressource |
| PUT | Modifikation einer Ressource |
| DELETE | Löschen einer Ressource |

13

Aktionen beschreiben gemäß dem REST-Paradigma Zustandsoperationen, die auf Ressourcen ausgeführt werden. Wichtig hierbei ist, dass die auszuführenden Aktionen zustandslos sind, das der Server behält keinen Zwischenzustand über die Anfragen des Clients hinweg vor. Stattdessen muss jede Anfrage alle notwendigen Kontextinformationen zur Durchführung mitbringen (eng. self-contained). Eine weitere Besonderheit ist, dass durch die Nutzung der HTTP-Anfragetypen generische Schnittstellen zur Veränderung der Ressourcen entstehen. Hierbei haben die HTTP-Methoden folgende übergreifende Bedeutung:

- GET -> Informationsbezug: Der Server erstellt eine Kopie der angefragten Ressource und sendet diese zum Client.
- POST -> Erstellt eine neue Ressource basierend auf den übergebenen Parametern.
- PUT -> Aktualisiert eine bestehende Ressource.
- DELETE -> Anweisung zum Löschen der referenzierten Ressource.

Die Aktionen GET, PUT und DELETE sind idempotent, das heißt unabhängig davon wie oft die jeweilige Abfrage durchgeführt wird, entsteht stets der gleiche Endzustand auf dem Server.

REST

Parameter Typen

Header-Parameter

GET POST PUT DELETE

Übergabe via Header, bspw. für Autorisierung & Authentifizierung, sowie Weiterleitung, **i.d.R. kein fachlicher Bezug**.

Bsp. für Header-Parameter

Accept-Encoding: gzip, deflate, br

Accept-Language: de,en-US;q=0.7,en;q=0.3

Connection: keep-alive

Path-Parameter

GET POST PUT DELETE

Bestandteil der aufgerufenen **URL**, beinhaltet Informationen zur **Identifikation der konkreten Ressource**.

http://my-corp.com/orders/1

Query-String Parameter

GET

Bestandteil der aufgerufenen **URL**, beschreibt Selektionskriterien zur Eingrenzung der zurückgegebenen Ressourcen.

http://my-corp.com/orders?geo=bw

Request-Body

POST PUT

Generell in JSON Notation. Beschreibung der zu erzeugenden oder zu verändernden Ressource.

```
{
  "OrderId": "2132",
  "CustomerId": "12321",
  "ProductId": "1232"
}
```

14

Für die Durchführung der REST-Aktionen werden folgende Arten von Parametern unterschieden:

- Header-Parameter können bei allen REST-Aktionen genutzt werden. Header-Parameter bestehen aus Schlüssel-Wert Paaren. Sie besitzen in der Regel keinen fachlichen Bezug sondern werden für Steuerungsinformationen wie Autorisierungs- und Authentifizierungsinformationen eingesetzt.
- Path-Parameter sind Bestandteil der aufgerufenen URL und beinhalten Informationen hinsichtlich der Identifikation der aufgerufenen Ressource. Path-Parameter können bei allen REST-Aktionen ausgewertet werden. Beispiele hierfür sind die Kunden-Nr. bei der Ausgabe von Detailinformationen zu einem Kunden.
- Query-String Parameter sind ebenso Bestandteil der aufgerufenen URL und beinhalten Abfragekriterien zur Eingrenzung der zurückgegebenen Ressourcen. Diese Art von Parameter wird primär bei GET-Abfragen eingesetzt. Query-String Parameter werden mit einem Fragezeichen an die URL angefügt.
- Abschließend wird der Request-Body bei POST und PUT Aktionen zur Übertragung von neu anzulegenden oder anzupassenden Objekten genutzt.

REST

Zustandsveränderungen

Ressource	POST	GET	PUT	DELETE
/movies	Neuen Film erstellen	Alle Filme zurückgeben	Massenaktualisierung aller Filme	Alle Filme entfernen
/movies/1	Fehler	Details für den Film mit der Id 1 abrufen	Details vom Film mit der Id 1 aktualisieren.	Film mit Id 1 entfernen

15

Die folgende Tabelle illustriert die unterschiedliche Bedeutung der HTTP-Abfragetypen anhand des Beispiels der Filmdatenbank. Unter Verwendung der Ressource ‚movies‘ wird die HTTP-POST Methode genutzt um einen neuen Film anzulegen. Eine GET-Abfrage auf dieser Ressource würde eine Liste aller gespeicherten Filme zurückgeben. Mit der HTTP-PUT Methode würde eine Massenaktualisierung aller Filme durchgeführt und der Aufruf der HTTP-DELETE Methode auf der Ressource würde zum Löschen aller Filme in der Datenbank führen.

Bei der Verwendung der Ressource /movies/1 (also der Auswahl eines spezifischen Films) würde die POST-Operation einen Fehler aufwerfen, da der Film bereits existiert und somit nicht neu angelegt werden kann. Der GET-Befehl würden die Detailinformationen zu diesem Film zurückgegeben. Die PUT-Operation würde diesen Film basierend auf den übergebenen Parametern aktualisieren und die DELETE-Aktion würde den referenzierten Film löschen.

REST

Request Mapping

- Eine REST-Schnittstelle wird in Spring Boot durch die Nutzung von **Annotationen** umgesetzt.
- **@RestController** spezifiziert, dass die Methoden des Controllers als REST-Schnittstelle umgesetzt werden.
- **@RequestMapping** spezifiziert den Kontext-Pfad in der URL der zwischen Domain und Ressource liegt.
- **@[GET/POST/PUT/DELETE]Mapping** bietet die darunterliegende Methode als REST-Aktion an.



Ausschnitte der Klasse MovieControllerImpl

16

Die Umsetzung einer REST-Schnittstelle in Spring Boot ist sehr komfortabel, da hier gemäß dem „convention-over-configuration“ Grundsatz automatisch sehr viele Best-Practice Konfigurationen gelten und nur Detailanpassungen via Annotationen vorgenommen werden müssen. Konkret werden folgende Annotation zur Umsetzung der REST-Aktionen eingesetzt:

- Die Annotation **@RequestMapping** über der Controller Klasse spezifiziert den Kontext-Pfad der **allen** Schnittstellenaufrufen des Rest-Controllers vorgestellt werden muss
- Das Anbringen der Annotation **@GetMapping** bietet die darunterliegende Methode als GET-Aktion nach außen an
- Über die Annotation **@PostMapping** und dem entsprechenden Ressourcenpfad wird die darunterliegende Methode als POST-Operation angeboten
- Die Annotation **@PutMapping** bietet die darunterliegende Methode als Put-Operation nach außen hin an.
- Die Annotation **@DeleteMapping** ermöglicht es die darunterliegende Methode nach außen hin als DELETE-Operation anzubieten.

REST

Parameter Typen

Es gibt vier unterschiedliche Arten der Parameterübergabe:

- **Header-Parameter**

Einsatz der Annotation **@RequestHeader** vor dem entsprechenden Parameter.

- **Path-Parameter**

Nutzung von geschweiften Klammern im Ressourcenpfad und **@PathVariable** vor dem Parameter in der Methodensignatur.

- **Query-String Parameter**

Verwendung der Annotation **@RequestParam** vor dem entsprechen Parameter in der Methodensignatur.

- **Request-Body**

Nutzung der Annotation **@RequestBody** vor dem entsprechenden Parameter in der Methodensignatur

Ausschnitte der Klasse *MovieControllerImpl*

```
@GetMapping
public List<Movie> getAllMovies(
    @RequestHeader String appId)
```

```
@DeleteMapping("/{movieId}")
public void deleteMovie(
    @PathVariable("movieId") Long movieId){
```

```
@GetMapping
public List<Movie> getMovies(
    @RequestParam String director){
```

```
@PostMapping
public Movie addMovie(
    @RequestBody Movie movie){
```

17

Schauen wir uns abschließend an wie die unterschiedlichen Parametertypen bei der Definition der Rest-Operationen mit Hilfe von Annotationen spezifiziert werden.

- Header-Parameter können bei allen HTTP-Anfragetypen über die Annotation **@RequestHeader** ausgelesen werden.
- Zum Auslesen der Path-Parameter müssen diese zum einen im Ressourcenpfad in geschweiften Klammern spezifiziert werden und zum anderen über die Annotation **@PathVariable** dem entsprechenden Parameter in der Methodensignatur zugewiesen werden.
- Query-String Parameter werden über die Annotation **@RequestParam** einem konkreten Parameter der Methode zugeordnet.
- Der Request Body kann mit der Annotation **@RequestBody** einem Objekt in der Methodensignatur zugewiesen werden.

REST

Hands-On

- Konfiguration des **Request Mappings**.
- Umsetzung der Methode **getAllMovies** als REST-Aufruf.
- Umsetzung der Methode **addNewMovie** als REST-Aufruf.
- Erläuterungen zu **Response-Code 200, 201 und 500**
- Aufruf der zwei REST-Methoden via **Postman**.

<http://localhost:8080/api/v1/movies/>

VS-Test / New Request

GET
▼
http://localhost:8080/api/v1/movies/

Params
Authorization
Headers (8)
Body ●
Pre-request Script
Tests
Settings

Body
Cookies
Headers (5)
Test Results

Pretty
Raw
Preview
Visualize
JSON ▼

```

1  {
2    "id": 1,
3    "title": "Interstellar",
4    "duration": 2.49,
5    "description": "A team of explorers travel through a wormhole i
6    "trailerUrl": "https://www.imdb.com/video/imdb/vi1586278169?pla
7    "coverImgUrl": "https://upload.wikimedia.org/wikipedia/en/b/bc/
8    "imdbRating": 8.6,
9    "castIds": [
10     4
11  ]

```

Aufgabe 4

RestController

Erstellen Sie bitte für den **AwardService** einen **RestController**, welcher die eingehenden Anfragen unter Nutzung des folgenden Schemas umsetzt. Verwenden Sie hierbei den **Kontext-Pfad „/api/v1/awards“**. Testen Sie Ihre Schnittstelle mit der Anwendung **Postman**

Ressource	POST	GET	PUT	DELETE
/awards	Nimmt eine Award Entität an und legt eine neue Ressource an.	Ausgabe aller Awards inkl. der verliehenen Filme	-	-
/awards/2	-	Ausgabe der Informationen zur Award Entität mit der Id 2	Aktualisierung der Award-Entität mit der Id 2	Löscht die Award Entität mit der Id 2



DTO

Grundlagen

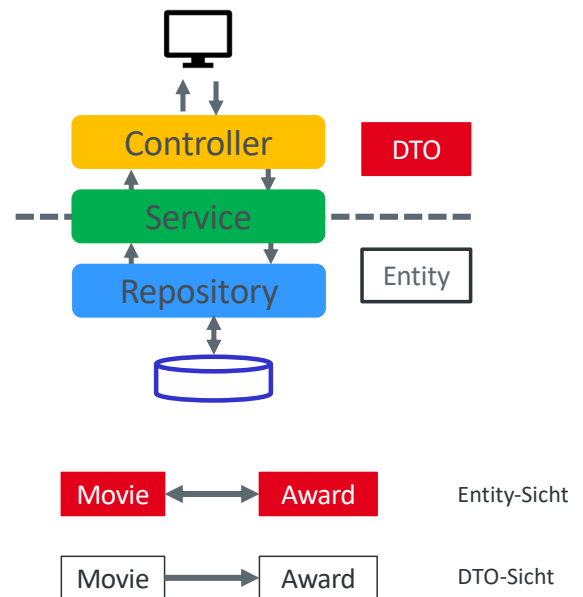
Datentransferobjekte (DTO) entkoppeln, die nach außen gegebenen Datenstrukturen von den intern verwendeten Datenstrukturen.

Vorteile:

- **Stabile API-Datenstruktur** trotz interner Weiterentwicklung.
- **Keine Weitergabe** von sensiblen Daten und bidirektionalen Referenzen.
- **Aggregation von Daten**, welche sonst mehrere Aufrufe erfordern.
- **Schutz vor Angriffen** auf interne Strukturen.

Herausforderung:

- DTOs sollten Entwickler **nicht unnötig behindern**.



Ein zentrales Problem, das aus der Weitergabe von Entitäten an Außenstehende resultiert ist die enge Kopplung der Darstellungssicht (Außensicht der Anwendung) mit der Implementierungssicht (Innensicht der Anwendung). Hierdurch können die internen Entitäten nicht weiterentwickelt werden ohne die Außensicht der Anwendung anzupassen. Des Weiteren ist es nicht möglich sensible Daten in der Außendarstellung zu verbergen oder Aggregationen zu nutzen, die es in der Innensicht nicht gibt. Auch angesichts von Sicherheitsaspekten sollte strikt vermeiden werden, dass Aufrufer interne-Datenstrukturen sehen, geschweige denn verändern, dürfen. Zu groß ist hier die Gefahr dass ein Angreifer diese Informationen für direkte (bspw. SQL-Injection) oder indirekte Angriffe (bspw. Ableitungen der Id) ausnutzt. In Folge dessen bietet es sich an die internen Datenstrukturen nicht direkt nach außen zu geben, sondern stattdessen Datentransferobjekte (DTO) als Abstraktion einzusetzen. Zentral für die Akzeptanz von DTOs ist, dass der Entwickler hierdurch nicht unnötig behindert wird, beispielsweise durch das Schreiben langer Getter-Setter-Wüsten.

DTO

MapStruct - Motivation

- Die Überführung der Entitäten in DTOs und der DTOs in Entitäten resultierte lange in einer **aufwändigen Konvertierungsarbeit** für die Entwickler.
- Die **manuelle Konvertierung** (Entität <-> DTO) ist sehr **fehleranfällig** und verursacht einen **laufenden Wartungsaufwand**.
- In der **Entwicklungspraxis** haben sowohl Entitäten als auch DTOs, häufig mehr **als 20 Attribute**.
- Gibt es eine Möglichkeit diese **Objekt-Konvertierung (Mapping)** **automatisiert** umzusetzen?

```
public MovieDTO convertToMovieDTO (Movie movie){
    MovieDTO movieDTO = new MovieDTO();
    movieDTO.setId(movie.getId());
    movieDTO.setTitle(movie.getTitle());
    movieDTO.setDescription(movie.getDescription());
    movieDTO.setDuration(movie.getDuration());
    movieDTO.setCoverImgUrl(movie.getCoverImgUrl());
    movieDTO.setImdbRating(movie.getImdbRating());
    return movieDTO;
}
```

Die manuelle Objekt-Konvertierung resultiert in wartungsreichem und fehleranfälligem Code. Lässt sich diese Konvertierung automatisieren?

Die Verwendung von DTOs resultierte lange Zeit in aufwändiger manueller Konvertierungsarbeit. Jedes Attribut musste hierbei über get-Methoden ausgelesen und über set-Methoden in das Zielobjekt geschrieben werden. Diese manuelle Konvertierung ist jedoch sehr fehleranfällig, da sehr schnell ein Attribut vergessen wird. Zudem resultiert eine manuelle Konvertierung stets in einem laufenden Wartungsaufwand, da die Konvertierung angepasst werden muss sobald Attribute in den DTOs oder in den Entitäten hinzukommen oder entfernt werden. Der Codeauszug zeigt, dass allein bei der Entität Movie in unserem Beispielprojekt viele Zeilen zur Konvertierung in ein MovieDTO notwendig sind (die Beziehungsattribute sind hierbei schon weggelassen). In der Praxis ist dieser Aufwand um ein Vielfaches höher, da die Entitäten hier generell mehr als 20 Attribute umfassen. Es stellt sich daher die dringende Frage ob die Objekt-Konvertierung DTO <-> Entität, die auch als Mapping bezeichnet wird, sich automatisieren lässt.

DTO

MapStruct – Grundlagen I

- **MapStruct** erzeugt **automatisch die Implementierung** für das Mapping DTO <-> Entität.
- Der Zusatz **componentModel = „spring“** ermöglicht es den Mapper wie eine Bean zu nutzen.
- **Lauten die Attribute** zwischen DTO und Entität **gleich**, wird nur das **Interface benötigt**. Andernfalls muss das Mapping der Attribute spezifiziert werden.
- Für das Mappen einer typisierten Liste muss auch das Mapping der Elemente definiert werden.
- Mit der Annotation **@InheritInverseConfiguration** leitet MapStruct automatisch die Gegenkonfiguration ab.

```
@Mapper(componentModel = "spring")
public interface AwardMapper {

    //Attribut movies wird automatisch ignoriert, da die Eigenschaft
    // nicht existiert
    @Mapping(source = "year", target = "celebrationYear")
    AwardDTO awardToAwardDTO(Award award);
    List<AwardDTO> awardsToAwardsDTO(List<Award> awards);

    @InheritInverseConfiguration
    Award awardDTOToAward(AwardDTO awardDTO);
    List<Award> awardDTOsToAwards(List<AwardDTO> awardDTOList);
}
```

Ausschnitt des Interface AwardMapper

23

MapStruct ist ein Framework, dass das Arbeiten mit DTOs massiv vereinfacht, indem es (ähnlich wie die No-Code Repositories) automatisch den Code für deren Konvertierung ableitet. Hierzu wird nur das Interface für das Mapping verfasst und mit der Annotation @Mapper ausgezeichnet. Mit dem Zusatz componentModel=„spring“ kann der Mapper wie eine reguläre Spring-Bean via Dependency Injection genutzt werden.

Für die Konvertierung Entity <-> DTO folgt MapStruct dem Grundsatz „convention over configuration“. Das heißt, dass sofern die Namen der Attribute in den DTO- und Entity-Objekten gleich lauten, keine weitere Konfiguration notwendig ist. Lauten die Attribute jedoch unterschiedlich muss manuell ein Mapping mit der Annotation @Mapping spezifiziert werden.

Für das Mapping von Listen muss ebenfalls das Mapping der einzelnen Listenobjekt spezifiziert werden. Eine sehr komfortable Funktion von MapStruct ist die Annotation @InheritInverseConfiguration wodurch MapStruct automatisch die Gegenseite des Mappings, basierend auf der bisherigen Konfiguration, ableitet.

DTO

MapStruct – Grundlagen II

- Ein Mapper, kann über den Parameter **uses**, die Vorschriften anderer Mapper „importieren“ um so geschachtelte Objekte zu unterstützen.
- Bei einer **Objekttyp-Umwandlung** wird der hierzu verwendende Code in die entsprechenden **map-Methode** geschrieben.

```
@Mapper(componentModel = "spring", uses = {AwardMapper.class})
public interface MovieMapper {

    @Mapping(target = "castIds", source = "involvedCast")
    MovieDTO movieToMovieDTO(Movie movie);
    List<MovieDTO> moviesToMovieDTOs(List<Movie> movies);

    Movie movieDTOToMovie(MovieDTO movieDTO);
    List<Movie> movieDTOsToMovies(List<MovieDTO> movieDTOs);

    default Long map(Cast cast) { return cast.getId(); }
```

Ausschnitt des Interface MovieMapper

MapStruct unterstützt neben der direkten Konvertierung der Klassenattribute auch die Konvertierung referenzierter Objekte. Zum Beispiel werden dem Movie-Objekt mehrere Award-Objekte zugeordnet. Damit auch die vom Movie-Objekt referenzierten Award-Objekte korrekt in DTOs konvertiert werden, ist es notwendig über den Zusatz „uses“ auf jene Mapper zu verweisen, die bei der Konvertierung eingebunden werden sollen. In unserem Beispiel würden so die referenzierten Award-Objekte eines Movie-Objekts gemäß dem AwardMapper in korrespondierende DTOs überführt.

Darüber hinaus kann der Entwickler selbst den Code für die Typ-Umwandlung einzelner Attributswerte verfassen, beispielsweise wenn Objektreferenzen in Fremdschlüssel-Beziehungen überführt werden sollen. In diesem Fall wird auf dem Interface die Methode map mit dem Eingangs- und Ausgangsdatentyp und der anzuwendenden Konvertierungsvorschrift spezifiziert. Damit die Methode zum Einsatz kommt muss zu Beginn der Zusatz „default“ angebracht werden.

Validierung

DTO Validierung

- Bei der **Objekt-Validierung** werden die Bedingungen in das zu überprüfende **Objekt** geschrieben.
- Zur Nutzung der Objektvalidierung muss bei der **Parameterdeklaration im Controller** die Annotation **@Valid** angebracht werden.
- Die **Struktur der Parametern** wird mithilfe von **@NotBlank**, **@NotNull**, **@Size**, **@Pattern** überprüft.
- Der **Wertebereich der Parameter** wird über die Annotationen **@Min** und **@Max** überprüft.
- Bei Verletzung einer Validierungsvorschrift wird eine **ConstraintViolationException** geworfen.

MovieControllerImpl

```
public Movie addMovie(@Valid @RequestBody MovieDTO movieDTO){
    return this.movieService.addMovie(movieDTO);
}
```

```
public class MovieDTO {
    private Long id;
    @NotBlank
    private String title;
    @Min(0)
    @Max(1000)
    private double duration;
    private String description;
}
```

MovieDTO

Mit der DTO Validierung existiert ein Mechanismus in Spring Boot der dazu dient die strukturellen Eigenschaften und Wertebereiche der übergebenen Parameter zu überprüfen. Durch die Nutzung dieses Mechanismus ist es möglich den Fokus des eigenen Codes auf die Umsetzung der Fachlichkeit zu legen und fertige Funktionen von Spring Boot zur Validierung der Parameter einzusetzen. Konkret wird bei der Objekt-Validierung ein ganzes Objekt validiert. Hierzu werden zunächst für die Attribute des Objekts die strukturellen Eigenschaften und Wertebereiche definiert und anschließend wird die Annotation **@Valid** im Methodenkopf vor Verwendung des jeweiligen Objektes platziert.

Zur Überprüfung der strukturellen Eigenschaften können folgende Annotationen eingesetzt werden: **@NotBlank** (stellt sicher dass das Attribut existiert und einen nichtleeren Wert besitzt), **@NotNull** (stellt sicher, dass das Attribut existiert), **@Size** (definiert für Collections die genaue Anzahl der Elemente), **@Pattern** (definiert den strukturellen Aufbau des Strings). Die Operationen **@Min** und **@Max** definieren die Minimal- und Maximalwerte für Double, Integer und Long Zahlenformate.

Validierung

Exception Handling

- Typische **Exceptions** die vom **Rest-Controller** gefangen und in eine strukturierte Antwort überführt werden sind:
 - **ConstraintViolationException** -> Fehler in der DTO Validierung.
 - **NoSuchElementException** -> Übergebene Id nicht in der Datenbank.
 - **EntityNotFoundException & EmptyResultDataAccessException** -> Übergebene Referenz nicht in der Datenbank.
- Die Fehlerrückmeldung an den Client erfolgt anschließend mit der **ResponseStatusException** und dem korrespondierenden HTTP-Antwortcode.

```
@PutMapping("/{movieId}")
public MovieDTO updateMovie(@PathVariable("movieId") Long movieId, @Valid
    movieDTO.setId(movieId);
    try {
        return this.movieService.updateMovie(movieDTO);
    } catch (EntityNotFoundException notFoundException) {
        throw new ResponseStatusException(HttpStatus.NOT_FOUND);
    } catch (NoSuchElementException noSuchElementException) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST);
    }
}
```

Ausschnitt der Klasse MovieControllerImpl

26

Ohne korrektes Exception Handling würde der Controller beim Auftreten von Exceptions stets den Statuscode 501 an den Client senden und dabei sogar mit dem Exceptiontext ggf. sensible Daten verraten. Ein korrektes Exception Handling ist daher für eine Web-Anwendung elementar. Um dies umzusetzen müssen in den Controllermethoden die auftretenden Exceptions gefangen und in strukturierte Fehlermeldungen überführt werden. Typische Exceptions die in den Controllermethoden gefangen werden sind die **ConstraintViolationException**, die bei Fehlern in der Validierung geworfen wird, die **NoSuchElementException**, die bei der Übergabe unbekannter Ids geworfen wird und die **EntityNotFoundException**, die geworfen wird, wenn eine Referenz nicht aufgelöst werden kann.

Die strukturierte Fehlermeldung, die an den Client zurückgegeben wird, erfolgt mithilfe der **ResponseStatusException** und der Auswahl des zurückgegebenen HTTP-Antwortcodes. Spring überführt anschließend die **ResponseStatusException** in eine strukturierte HTTP-Antwort.

Dokumentation

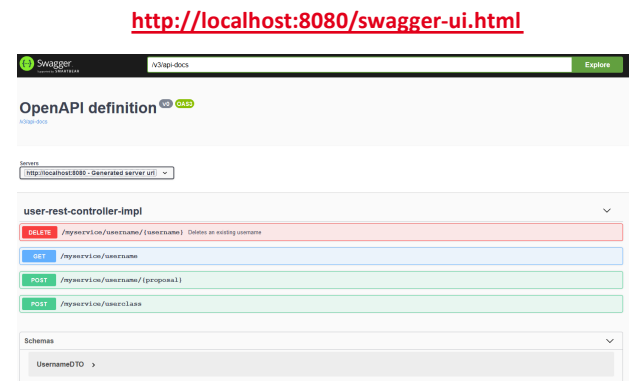
Grundlagen

OpenAPI ist das **populärste Framework** zur Dokumentation von REST-Schnittstellen.

Die OpenAPI Dokumentation ist das **Bindeglied** für die Kooperation von Front-End und Back-End Teams.

Spring wertet den **REST-Controller** und die **Annotation aus**, um das Grundgerüst für die OpenAPI Dokumentation zu erstellen.

Auch **Validierungsbedingungen** werden automatisch in die OpenAPI Dokumentation eingebettet.



OpenAPI ist der de-facto Standard zur Dokumentation von REST-Schnittstellen. Ein wichtiger Vorteil von OpenAPI ist, dass das verwendete Dokumentationsformat sprachunabhängig ist, sodass es plattformübergreifend verwendet werden kann. Der OpenAPI Dokumentation kommt im Unternehmenseinsatz eine wichtige Rolle zu, da sie oft das Bindeglied zwischen Front-End und Back-End Entwicklern darstellt.

Die Integration von OpenAPI in Spring ist sehr komfortabel umgesetzt. So werden automatisch die eingesetzten Annotationen des Rest-Controllers ausgewertet um ein Grundgerüst der OpenAPI Dokumentation abzuleiten. Ferner werden die Validierungsbedingungen ausgewertet und ebenfalls automatisch in die Dokumentation übernommen.

Die OpenAPI-Dokumentation einer laufenden Spring-Applikation kann unter der Adresse <http://localhost:8080/swagger-ui.html> aufgerufen werden.

Dokumentation

Spring

Die OpenAPI Dokumentation kann durch die Verwendung von **spezifischen Annotationen** ausgebaut werden.

Weitere Ergänzungen sind:

- Die Beschreibung der **erbrachten Funktionalität** in Kurz- und Langform.
- Die **fachliche Bedeutung** der verwendeten **HTTP-Antwortcodes**.
- **Beschreibung** der Bedeutung von **Request- und Pfad-Parametern**.

Ausschnitt der Klasse MovieControllerImpl

```
@Operation(summary = "Adds a new movie to the database", description = "This operation " +
    "creates and persists the given movie in the database")
@ApiResponses(value = {
    @ApiResponse(responseCode = "201", description = "The movie has been successfully added."),
    @ApiResponse(responseCode = "400", description = "The passed movie details are not valid"),
    @ApiResponse(responseCode = "500", description = "An error occurred during processing")
})
@PostMapping
public MovieDTO addMovie(@Parameter(description = "The new movieDTO which will be stored.")
```

```
@PutMapping("/{movieId}")
public MovieDTO updateMovie(@Parameter(description = "The unique movieId")
    @PathVariable("movieId") Long movieId,
    @Parameter(description = "The updated movieDTO")
    @Valid @RequestBody MovieDTO movieDTO){
    movieDTO.setId(movieId);
    try {
        return this.movieService.updateMovie(movieDTO);
    } catch (EntityNotFoundException notFoundException) {
```

Ausschnitt der Klasse MovieControllerImpl

Neben der automatischen Generierung des OpenAPI-Grundgerüsts, kann die OpenAPI Dokumentation mithilfe von Annotationen im RestController ergänzt werden. So kann mit der Annotation `@Operation` die Funktionalität der HTTP-Schnittstelle beschrieben werden. Des Weiteren werden mit der Annotation `@ApiResponse` die HTTP-Statuscodes, sowie deren fachspezifische Interpretation beschrieben. Abschließend werden Parameter sowohl inline als auch im jeweiligen Objekt mit der Annotation `@Parameter` fachlich beschrieben.

Aufgabe 5

Controller Test & DTO

- a) Erstellen Sie den **AwardMapper**, ein Mapper für den **AwardDTO**, sowie eine Liste von **AwardDTO**. Mappen Sie hierbei Award.year auf AwardDTO.celebrationYear. Testen Sie Ihre Implementierung indem Sie:
 - Den Mapper in der **getAllAwards** Methode im **AwardServiceImpl** verwenden und
 - Mithilfe von **Postman** die Methode **getAllAwards aufrufen** und die Antwort überprüfen
- b) Stellen Sie mithilfe der **Objektvalidierung** sicher, dass der übergebene AwardDTO der Methode **addAward** einen Kategorienamen besitzt. Ist dies nicht der Fall geben Sie eine **ResponseStatusException** mit dem **HTTP-Code 400 (Bad Request)** zurück.
- c) Erweitern Sie die **OpenAPI Dokumentation** für die Methode **updateAward** in **AwardControllerImpl**.